

K8s Poka-Yoke — Complete End-to-End Blueprint

Final project positioning

K8s Poka-Yoke: Policy-Enforced GitOps & Zero-Downtime Kubernetes Deployment Engine

The core idea:

Prevent bad application code in CI/CD, prevent unsafe workloads at Kubernetes admission, continuously reconcile infrastructure through GitOps, and automatically recover from runtime failures.

Your existing Poka-Yoke project handles the first half:

```
Code
↓
GitHub Actions
↓
Tests / Security / Terraform validation
↓
❌ Bad change blocked
```

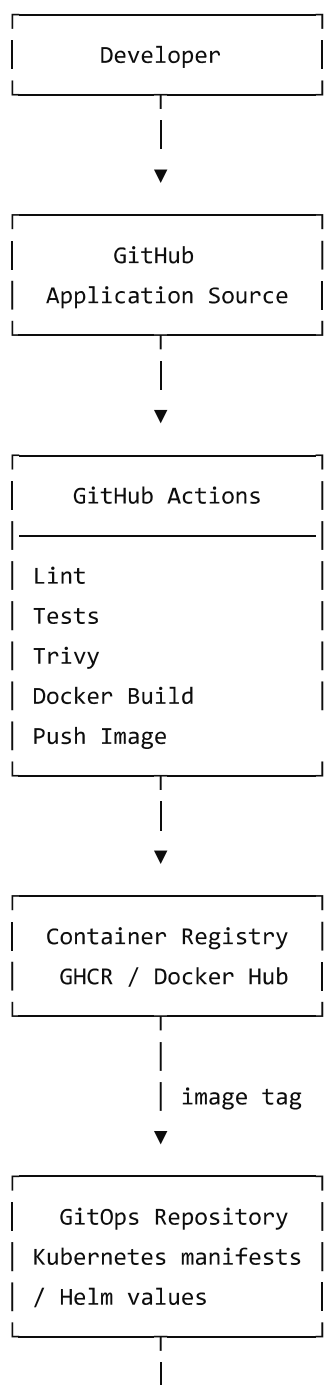
This new project handles the Kubernetes side:

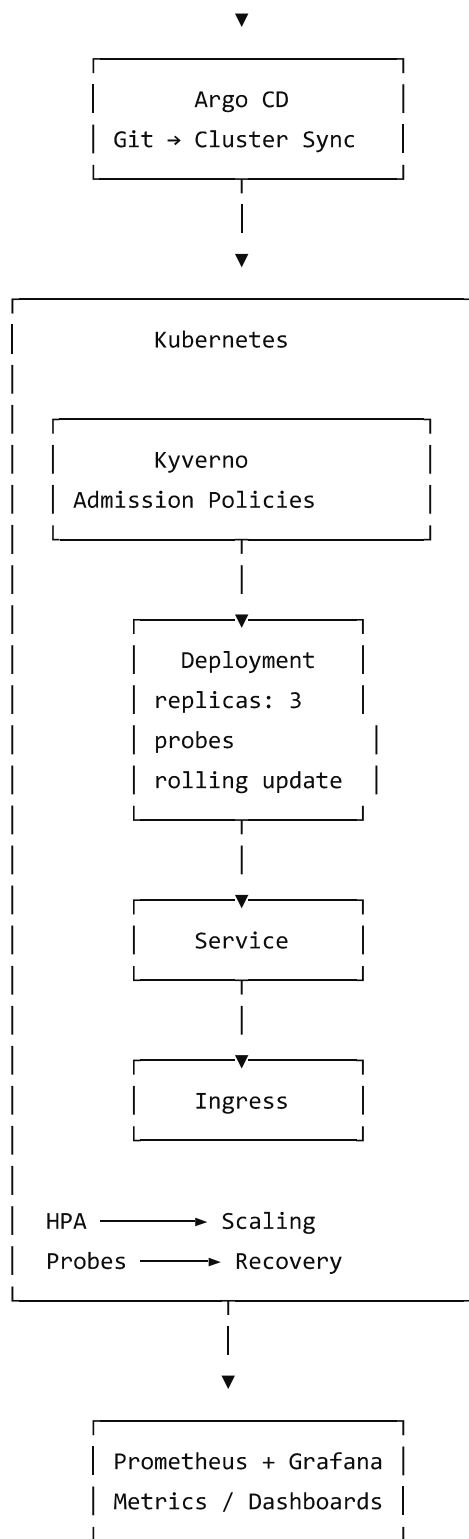
```
Container
↓
GitHub Actions
↓
Container Registry
↓
GitOps Repository
↓
Argo CD
↓
Kubernetes API
↓
Kyverno
↓
❌ Unsafe workload blocked
↓
✅ Safe workload deployed
↓
Probes + HPA + Rolling Update
↓
```

PHASE 0 — Define the architecture

Before writing YAML, define what you're building.

Architecture





PHASE 1 — Repository foundation

Start with:

k8s-poka-yoke/

```
|
├─ app/
|   ├── src/
|   |   └─ main.py
|   ├── requirements.txt
|   └─ Dockerfile
|
├─ k8s/
|   ├── base/
|   |   ├── namespace.yaml
|   |   ├── deployment.yaml
|   |   ├── service.yaml
|   |   ├── configmap.yaml
|   |   ├── ingress.yaml
|   |   ├── hpa.yaml
|   |   └─ serviceaccount.yaml
|   ├── policies/
|   |   ├── disallow-privileged.yaml
|   |   ├── require-resources.yaml
|   |   ├── disallow-latest-tag.yaml
|   |   ├── require-non-root.yaml
|   |   ├── require-probes.yaml
|   |   └─ require-readonly-rootfs.yaml
|   └─ tests/
|       ├── bad-privileged.yaml
|       ├── bad-latest.yaml
|       ├── bad-resources.yaml
|       └─ compliant.yaml
|
├─ argocd/
|   ├── application.yaml
|   └─ project.yaml
|
├─ monitoring/
|   ├── servicemonitor.yaml
|   └─ dashboards/
|
├─ tests/
|   ├── smoke/
|   └─ failure/
|
├─ .github/
|   ├── workflows/
|   |   ├── ci.yaml
|   |   └─ security.yaml
```

```
|
├─ kind-cluster.yaml
├─ Makefile
└─ README.md
```

Don't worry if the exact structure changes later. This is the **initial architecture**, not a rule carved in stone.

PHASE 2 — Build the application

You need an application that can demonstrate Kubernetes behavior.

Don't make it just:

```
GET /
Hello World
```

Give it endpoints specifically designed for reliability testing.

For example:

```
GET /
GET /healthz
GET /ready
GET /metrics
POST /kill
GET /version
```

The application should expose:

```
/
```

Return:

```
{
  "status": "online",
  "service": "k8s-poka-yoke",
  "version": "1.0.0",
  "pod": "..."
```

```
/healthz
```

Liveness check.

`/ready`

Readiness check.

Initially:

503

Then after initialization:

200

This lets you demonstrate that Kubernetes doesn't send traffic to an application before it's ready.

`/kill`

Purposefully put the application into an unhealthy state.

This is extremely useful later.

You can demonstrate:

```
healthy pod
  ↓
POST /kill
  ↓
liveness probe fails
  ↓
Kubernetes restarts container
  ↓
application becomes healthy again
```

That's much more impressive than simply saying "Kubernetes is self-healing."

PHASE 3 — Secure container

Build your Docker image correctly.

Your container should:

- use a small base image

- avoid running as root
- pin dependencies
- expose only required port
- avoid unnecessary packages
- use a non-root UID
- eventually use a read-only filesystem

Then scan it:

```
trivy image <image>
```

Your existing Poka-Yoke project already gives you a strong DevSecOps foundation here.

PHASE 4 — Create the local Kubernetes environment

Use `kind` first.

Why?

Because your objective is learning and demonstrating Kubernetes/GitOps—not spending the first week debugging AWS networking.

Cluster:

```
control-plane  
worker  
worker
```

Verify:

```
kubectl get nodes  
kubectl get pods -A
```

Then install:

- Kyverno
- Metrics Server
- Argo CD

- Prometheus
- Grafana
- Ingress controller

Install them **one at a time**, verifying each before moving on.

PHASE 5 — Kubernetes application deployment

Now create the actual workload.

Deployment

Target:

```
3 replicas
```

Configure:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 0
```

This gives you your first zero-downtime demonstration.

Also configure:

```
resources.requests
resources.limits
```

and:

```
livenessProbe
readinessProbe
startupProbe
```

This is where the project starts looking like a production Kubernetes workload rather than a tutorial deployment.

PHASE 6 — Service networking

Create a Kubernetes Service.

Flow:

Ingress

↓

Service

↓

Pod 1

Pod 2

Pod 3

Demonstrate:

```
kubectl get svc  
kubectl get endpoints
```

Then kill one Pod:

```
kubectl delete pod <pod>
```

Watch:

```
kubectl get pods -w
```

Expected:

Pod disappears

↓

Deployment notices replica shortage

↓

New Pod created

↓

New Pod becomes Ready

↓

Service includes it

That's another important portfolio demonstration.

PHASE 7 — Kyverno admission control

Now your project's **Poka-Yoke engine** begins.

Start with the three policies Gemini gave you:

Policy 1

No privileged containers.

```
privileged: true
```



✖ REJECTED

Policy 2

Resources mandatory.

```
requests + limits missing
```



✖ REJECTED

Policy 3

No `:latest`.

```
nginx:latest
```



✖ REJECTED

Then add stronger policies.

Policy 4 — Non-root

Require:

```
runAsNonRoot: true
```

Demonstration:

```
runAsNonRoot missing/false
```



✖ REJECTED

Policy 5 — Read-only filesystem

Require:

```
readOnlyRootFilesystem: true
```

This strengthens the container security model.

Policy 6 — Probes

Require workloads to define:

```
startupProbe
readinessProbe
livenessProbe
```

Now you're enforcing operational reliability at admission.

PHASE 8 — Build the Poka-Yoke test suite

This is **very important**.

Don't merely have policies.

Have reproducible tests proving that the policies work.

Create:

```
tests/
├─ bad-privileged.yaml
├─ bad-resources.yaml
├─ bad-latest.yaml
└─ bad-root.yaml
```



Test:

```
kubectl apply -f tests/bad-privileged.yaml
```

Expected:

```
✖ admission denied
```

Then:

```
kubectl apply -f tests/compliant.yaml
```

Expected:

```
✔ created
```

Your README can eventually contain a table:

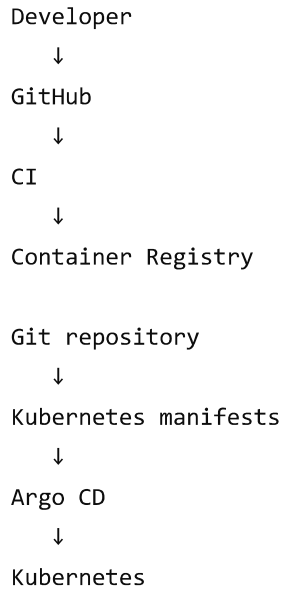
Test	Expected	Result
Privileged container	Reject	✔
Missing resources	Reject	✔
:latest	Reject	✔
Root container	Reject	✔
Missing probes	Reject	✔
Compliant workload	Accept	✔

That's portfolio evidence.

PHASE 9 — Argo CD GitOps

Now introduce GitOps.

The desired architecture becomes:



Install Argo CD.

Create an Argo CD Application pointing at your Git repository.

Configure:

```
automated sync
prune
selfHeal
```

Conceptually:

```
graph TD; Git["Git = desired state"] --> Argo_CD[Argo CD]; Argo_CD --> Cluster["Cluster = actual state"];
```

The conceptual diagram shows the reconciliation loop. Git represents the 'desired state'. Argo CD continuously compares the actual state of the Kubernetes cluster with the desired state in Git. If there is a discrepancy, Argo CD initiates a sync to bring the cluster's state into alignment with the desired state.

PHASE 10 — Demonstrate GitOps reconciliation

This is one of your strongest demos.

Deploy:

```
version 1.0.0
```

Then manually change the live cluster:

```
kubectl scale deployment ... --replicas=1
```

But Git says:

```
replicas: 3
```

Argo CD detects drift.

Then:

```
Cluster
  replicas = 1
    ↓
Argo CD detects drift
    ↓
Git says replicas = 3
    ↓
Argo CD reconciles
    ↓
replicas = 3
```

That is **real GitOps behavior**.

Capture it.

PHASE 11 — CI/CD pipeline

Now connect GitHub Actions.

Pipeline:

```
git push
  ↓
Checkout
  ↓
Lint
```





```
graph TD; A[ ] --> B[Unit tests]; B --> C[Security scan]; C --> D[Docker build]; D --> E[Trivy image scan]; E --> F[Push image]; F --> G[Update Kubernetes image tag]; G --> H[Git commit]; H --> I[Argo CD detects change]; I --> J[Deployment];
```

↓
Unit tests
↓
Security scan
↓
Docker build
↓
Trivy image scan
↓
Push image
↓
Update Kubernetes image tag
↓
Git commit
↓
Argo CD detects change
↓
Deployment

Important distinction:

GitHub Actions should not directly `kubectl apply` **your production manifests.**

For the GitOps design:

CI → changes Git
CD → Argo CD changes cluster

That separation is worth discussing in interviews.

PHASE 12 — Image versioning

Do **not** use:

`latest`

Use immutable versions.

For example:

`ghcr.io/rahul/k8s-poka-yoke:v1.0.0`
`ghcr.io/rahul/k8s-poka-yoke:v1.1.0`

Even better, eventually demonstrate digest-based deployment.

Your Kyverno policy then reinforces this:

latest



v1.2.0



PHASE 13 — Zero-downtime deployment

This becomes one of your flagship demonstrations.

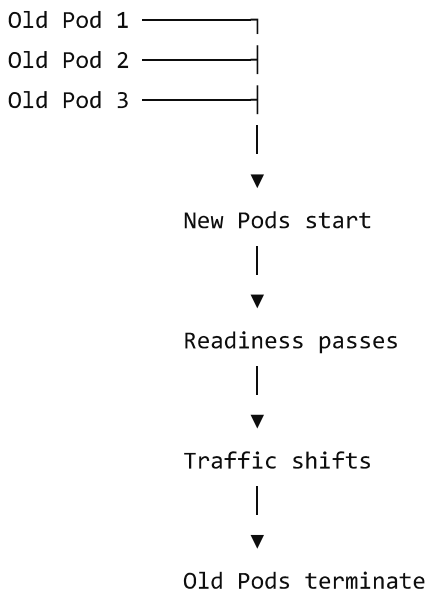
Run continuous traffic against:

v1.0.0

Then push:

v1.1.0

Kubernetes performs:



Your configuration:

```
maxUnavailable: 0
```

supports the intended availability behavior.

Then prove it with requests:

```
request 1 → 200  
request 2 → 200  
request 3 → 200  
...
```

during deployment.

PHASE 14 — HPA

Install Metrics Server.

Create HPA:

```
minimum replicas = 2  
maximum replicas = 10  
CPU target = 60%
```

Then generate load.

Conceptually:

```
Normal load  
↓  
2 Pods  
  
High CPU  
↓  
HPA detects utilization  
↓  
4 Pods  
  
Higher load  
↓  
6 Pods
```

Then stop the load:

```
6 Pods
↓
CPU falls
↓
HPA
↓
2 Pods
```

Now your project demonstrates **automatic scaling**, not just static replicas.

PHASE 15 — Failure injection

This is where your project becomes genuinely interesting.

Create deliberate failure scenarios.

Failure A — Application crash

```
POST /kill
```

Then:

```
liveness probe fails
↓
container restarted
↓
application recovers
```

Failure B — Pod deletion

```
kubectl delete pod <pod>
```

Expected:

```
Deployment → replacement Pod
```

Failure C — Readiness failure

Make the application return:

```
503
```

on readiness.

Expected:

```
Pod still exists  
BUT  
Service stops sending traffic to it
```

That's an important Kubernetes distinction.

Failure D — Bad deployment

Push an intentionally invalid manifest.

For example:

```
privileged: true
```

Expected:

```
Git  
↓  
Argo CD  
↓  
Kubernetes API  
↓  
Kyverno  
↓  
✗ rejected
```

That's the actual Poka-Yoke moment.

PHASE 16 — Observability

Install:

Prometheus
Grafana

Collect:

- CPU
- memory
- pod count
- container restarts
- request metrics
- HPA activity
- deployment status

Build a Grafana dashboard showing at minimum:

Pod Count
CPU Utilization
Memory Utilization
Container Restarts
Request Rate

Then deliberately kill a Pod and show the restart/availability behavior on the dashboard.

PHASE 17 — Ingress

Add an Ingress.

Flow:

Browser
↓
Ingress
↓
Service



Use a hostname locally, for example:

poka-yoke.local

You can later add TLS.

Don't spend excessive time trying to build a sophisticated production ingress stack locally. The goal is demonstrating Kubernetes networking and routing.

PHASE 18 — Security hardening

At this stage, strengthen the workload.

Your final workload should aim for:

Non-root



No privileged container



Read-only root filesystem



Resource requests/limits



Explicit image version



SecurityContext



ServiceAccount



RBAC



NetworkPolicy



Add a NetworkPolicy such as:

```
graph TD; Frontend --> Backend1[Backend]; Backend1 --> Backend2[Backend]; Backend2 --> Database;
```

and prevent arbitrary pod-to-pod communication.

That gives you another excellent interview topic.

PHASE 19 — RBAC

Don't run everything with excessive privileges.

Create a dedicated ServiceAccount.

Demonstrate:

```
graph TD; Application --> ServiceAccount; ServiceAccount --> RBAC; RBAC --> Permissions[Only required permissions];
```

You should be able to explain:

Authentication tells Kubernetes who you are; authorization through RBAC determines what you are allowed to do.

PHASE 20 — Automated end-to-end testing

Now automate the demonstrations.

Create something like:

```
tests/
├─ admission/
│   ├─ privileged_test.sh
│   ├─ resources_test.sh
│   ├─ latest_test.sh
│   └─ nonroot_test.sh
├─ reliability/
│   ├─ pod_recovery.sh
│   ├─ readiness_test.sh
│   └─ rolling_update.sh
└─ gitops/
    └─ drift_reconciliation.sh
```

Eventually:

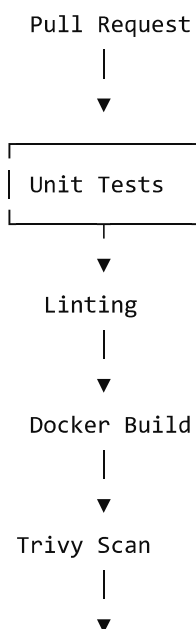
```
make test
```

should execute your important checks.

That is much stronger than manually running 30 commands during a demo.

PHASE 21 — GitHub Actions quality gate

Your CI pipeline should eventually perform:



```
graph TD; A[Kubernetes YAML] --> B[Validation]; B --> C[Policy Testing]; C --> D[PASS];
```

A bad PR should fail before it reaches GitOps.

PHASE 22 — Optional advanced layer: Policy testing in CI

This is where the project becomes particularly strong.

You have:

Runtime/admission protection

```
graph TD; A[Kyverno] --> B[Cluster rejects unsafe workload];
```

And:

CI protection

```
graph TD; A[GitHub Actions] --> B[Policy validation]; B --> C[Bad manifest rejected before merge];
```

So now you have **defense in depth**.

```
graph TD; A[Developer] --> B[CI security gates]; B --> C[Git]; C --> D[Argo CD]; D --> E[Kyverno admission];
```



That's an excellent architecture to explain.

PHASE 23 — Optional AWS/EKS phase

Do this last.

Not at the beginning.

Once the local platform works:

```
kind
  ↓
complete project
  ↓
EKS
```

Then you can reproduce the architecture on AWS.

Potential AWS components:

```
AWS
|
├─ EKS
├─ VPC
├─ IAM
├─ ECR
├─ ALB
└─ CloudWatch
```

Your existing resume already has substantial AWS work, so **EKS should be the extension, not the core project.**

If you finish the local implementation first, you'll learn Kubernetes much faster.

PHASE 24 — Infrastructure as Code

Once the application works manually, Terraform the infrastructure where appropriate.

For an EKS extension:

```
terraform/  
├─ vpc/  
├─ eks/  
├─ iam/  
├─ ecr/  
└─ variables.tf
```

Don't Terraform every local kind object just for the sake of saying "Terraform."

Use Terraform where it provides meaningful infrastructure automation.

PHASE 25 — Final failure matrix

Your final project should be able to demonstrate something like this:

Scenario	Mechanism	Expected behavior
Privileged container	Kyverno	 Rejected
Missing CPU/memory	Kyverno	 Rejected
:latest	Kyverno	 Rejected
Root container	Kyverno	 Rejected
Missing probes	Kyverno	 Rejected
Compliant workload	Kyverno	 Accepted
Pod crashes	Liveness probe	 Restart
Pod deleted	Deployment	 Replacement
Pod not ready	Readiness probe	 Removed from traffic
High CPU	HPA	 Scale out
Low CPU	HPA	 Scale in
Git drift	Argo CD	 Reconciled

Scenario	Mechanism	Expected behavior
New version	RollingUpdate	 Zero-downtime rollout
Bad Git change	CI	 Pipeline fails
Bad Kubernetes workload	Kyverno	 Admission fails
Unauthorized network access	NetworkPolicy	 Blocked

That table is essentially your project's acceptance criteria.

PHASE 26 — Documentation

Don't leave documentation until the last hour.

Your README should eventually have:

```
# K8s Poka-Yoke

## Problem

## Solution

## Architecture

## Technology Stack

## Repository Structure

## Prerequisites

## Local Setup

## Application

## Kubernetes Architecture

## Kyverno Policies

## GitOps with Argo CD

## CI/CD

## HPA
```

Observability

Security

Failure Testing

Results

Screenshots

Demo

Lessons Learned

Future Improvements

Most importantly, document **why** you made decisions.

For example:

Why Argo CD instead of direct kubectl deployment?

Why admission control instead of CI-only validation?

Why readiness and liveness are separate?

Why `maxUnavailable: 0` ?

Why immutable image tags?

Why resource requests and limits?

Those are interview questions waiting to happen.

PHASE 27 — Evidence you should collect

By the end, your GitHub repository should contain evidence such as:

1. Architecture diagram

GitHub → Actions → Registry → Argo CD → Kubernetes



Kyverno



Deployment / HPA



Prometheus/Grafana

2. Kyverno rejection screenshot

✖ POKA-YOKE REJECTED

3. Successful deployment

kubectl get pods

3/3 Running

4. Argo CD screenshot

Show:

Synced
Healthy

5. Git drift recovery

OutOfSync



Sync



Healthy

6. HPA scaling

2 → 4 → 6 pods

7. Failure recovery

Pod crash



restart



healthy

8. Zero-downtime rollout

Show old and new versions transitioning while traffic continues.

9. Grafana dashboard

Show actual metrics.

PHASE 28 — Final project structure

After everything is mature, I would aim for something approximately like:

```
k8s-poka-yoke/
|
├─ app/
|   ├── src/
|   |   └─ main.py
|   ├── requirements.txt
|   └─ Dockerfile
|
├─ k8s/
|   ├── base/
|   |   ├── namespace.yaml
|   |   ├── deployment.yaml
|   |   ├── service.yaml
|   |   ├── configmap.yaml
|   |   ├── ingress.yaml
|   |   ├── hpa.yaml
|   |   ├── serviceaccount.yaml
|   |   ├── role.yaml
|   |   ├── rolebinding.yaml
|   |   └─ networkpolicy.yaml
|   |
|   └─ policies/
|       ├── disallow-privileged.yaml
|       ├── require-resources.yaml
|       ├── disallow-latest-tag.yaml
|       ├── require-non-root.yaml
|       ├── require-probes.yaml
|       └─ require-readonly-rootfs.yaml
|
└─ tests/
```

```
|      |─ bad-privileged.yaml
|      |─ bad-resources.yaml
|      |─ bad-latest.yaml
|      |─ bad-root.yaml
|      └─ compliant.yaml
|
|─ argocd/
|  |─ project.yaml
|  └─ application.yaml
|
|─ monitoring/
|  |─ servicemonitor.yaml
|  └─ dashboards/
|
|─ tests/
|  |─ admission/
|  |─ reliability/
|  └─ gitops/
|
|─ .github/
|  └─ workflows/
|     |─ ci.yaml
|     └─ security.yaml
|
|─ terraform/
|  └─ eks/
|     └─ ...
|
|─ kind-cluster.yaml
|─ Makefile
└─ README.md
```

The actual milestone roadmap

So don't think of this as "Milestone 1 = project."

Think:

```
M0  Architecture & acceptance criteria
    ↓
M1  Kind + application + Docker
    ↓
M2  Kubernetes Deployment + Service
    ↓
M3  Probes + self-healing + rolling updates
```

↓
M4 Kyverno admission control
↓
M5 Admission failure-test suite
↓
M6 Argo CD GitOps
↓
M7 GitOps drift/reconciliation
↓
M8 GitHub Actions CI/CD
↓
M9 Immutable image versioning
↓
M10 HPA + Metrics Server
↓
M11 Ingress
↓
M12 Prometheus + Grafana
↓
M13 RBAC + NetworkPolicy + hardening
↓
M14 Automated E2E/failure testing
↓
M15 Complete documentation + architecture diagram
↓
M16 Demo video + portfolio evidence
↓
M17 Optional EKS + Terraform

And there is a very important rule
Don't build all 17 milestones simultaneously.

Build vertically.

At each stage you should have something that works:

M1:
Container runs

M2:
Container runs in Kubernetes

M3:
Kubernetes automatically handles failure

M4:
Kubernetes rejects unsafe workloads



```
M6:  
Git controls Kubernetes  
  
M8:  
GitHub automatically builds/deloys  
  
M10:  
Kubernetes automatically scales  
  
M12:  
You can observe the system  
  
M14:  
The project automatically proves its guarantees
```

That last part is what makes this different from a typical fresher project.

Your existing **Poka-Yoke DevSecOps Gatekeeper** says:

"I prevent bad software changes from passing CI."

This project should say:

"I prevent unsafe Kubernetes workloads from entering the cluster, continuously reconcile desired state through GitOps, and demonstrate self-healing, scaling, zero-downtime deployment, and observability."

That gives you a **coherent two-project portfolio story**, rather than two unrelated projects.

Would you like me to turn this blueprint into a week-by-week implementation plan or a complete repository starter structure?